

Towards **Security Commit** **Message** Standardization

by



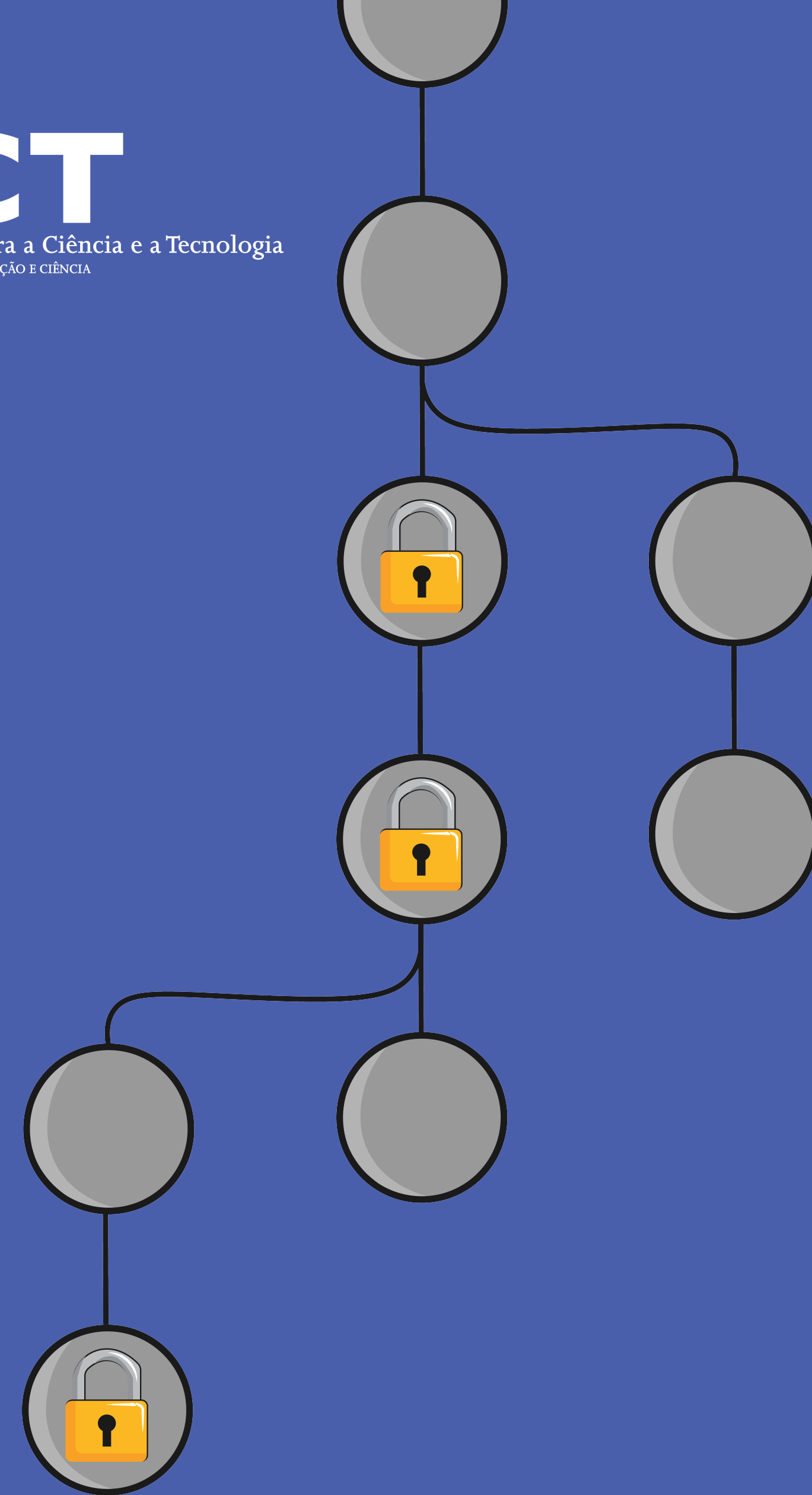
Sofia Reis
INESC-ID & IST/Técnico
University of Lisbon
Portugal



Rui Abreu
INESC-ID & FEUP
University of Porto
Portugal



Corina Păsăreanu
Carnegie Mellon
University
USA



Why focus on security commit messages?



Timely patch management prevents attacks. However, efficient patch triage systems remain a challenge.



Previous research has investigated using **commit messages** and/or **code changes** for automated patch detection.



62% of security fixes are *silent* – no clear message. i.e., do not include any security-related word!



Poor documentation delays triage and patch deployment and commit messages are the **first** and **most direct** form of documentation for security fixes.

Motivation

TABLE I: Examples of Real Security Commit Messages

#	Commit Message	Vulnerability ID	SHA
1	.	CVE-2019-13568	ac80033
2	Minor patch.	OSV-2020-2108	a8bf10e
3	Code refactoring.	GHSA-4fc4-4p5g-6w89	d158413
4	<scratchsig><script>location='https://www.youtube.com/watch?v=dQw4w9WgXcQ';</script></scratchsig>	CVE-2020-15179	4160a39
5	Fixed security issues with passwords entered via a prompt	CVE-2022-35928	6876185

😂 Message 4 is an internet meme known as "Rickrolling," where people are pranked with the song "Never Gonna Give You Up"—humorous, but it provides no relevant information about the patch.

Research Questions



R1. Which types of information can be extracted from security-related commit messages?



RQ2. How do security engineers apply best practices when writing security commit messages?



RQ3. How do LLMs influence the completeness of security commit messages?

Study Methodology

R1 (Completeness) & R2 (Structure)



Analyzed 11,036 security commit messages.
Data collected from OSV and NVD.



Built a custom **Named Entity Recognition (NER) tool** to extract security-related data.



Evaluated adherence to best practices for writing commit messages.
Conventional Commits, Linus Torvalds, Chris Beams, Tian et al. and Atomic Changes

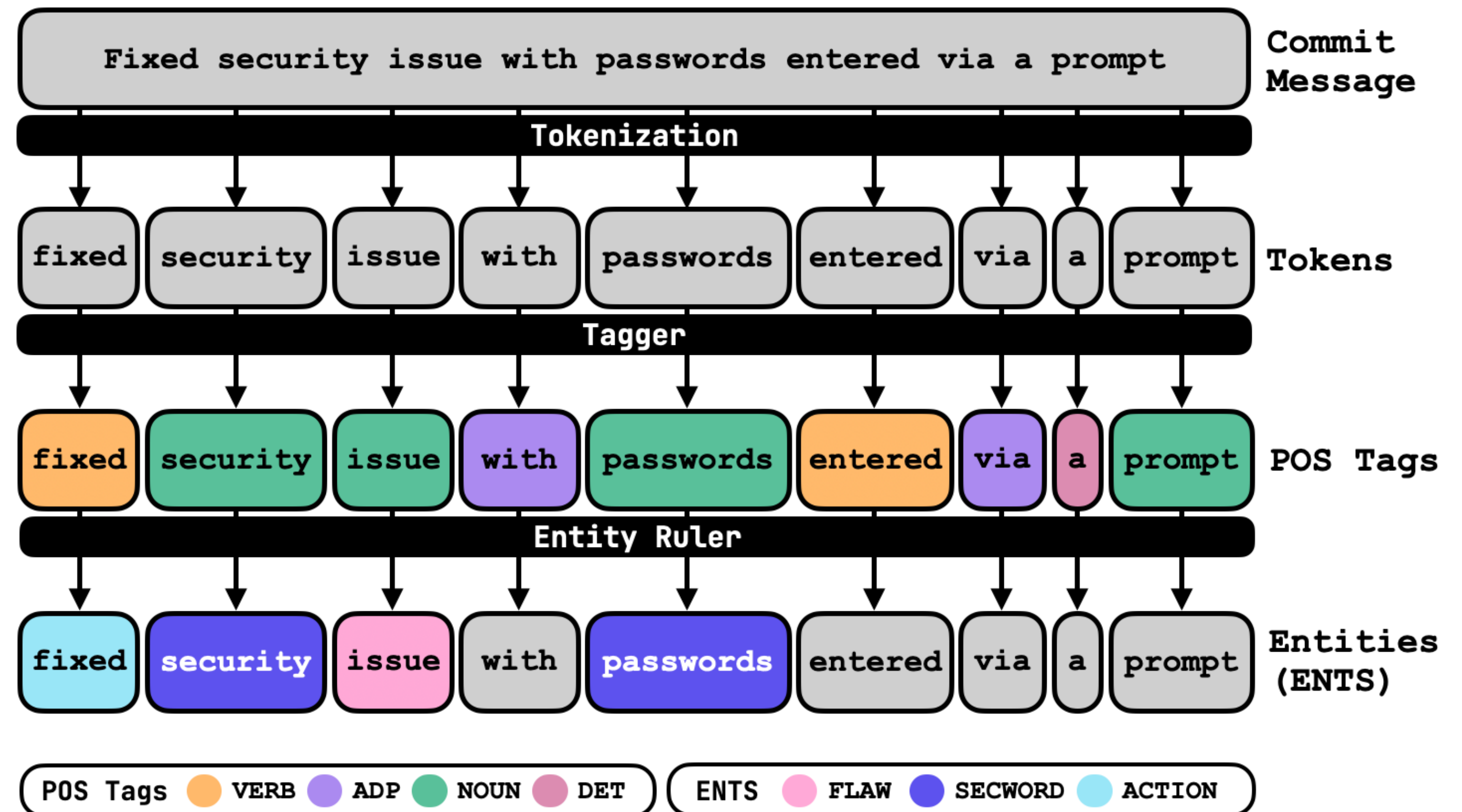
NER for Security and Commit Data

Security Specific Categories

- SECWORD, VULNID, CWEID, SEVERITY, DETECTION

Commit Specific Categories

- SHA, ACTION, FLAW, ISSUE, EMAIL, URL, VERSION



NER Rules

1789 rules to extract 12 different types of categories

TABLE II: Category names, rationale, and entity examples

Type	Category	Rationale	Entity Examples	Rules
SEC	SECWORD	Security-relevant terms are typically used to describe both the vulnerability and its corresponding fix.	ldap injection, crlf injection, improper validation, command injection, cross-site scripting	1718
	VULNID	Vulnerability IDs are used to identify vulnerabilities for different ecosystems in commit messages: CVE, GHSA, OSV, PyPI, etc.	GHSA-269q-hmxg-m83q, CVE-2016-2512, OSV-2016-1	16
	CWEID	Vulnerabilities are typically associated with a specific type of weakness from the Common Weakness Enumeration (CWE) taxonomy.	CWE-119, CWE-20, CWE-79, CWE-189	6
	SEVERITY	Vulnerabilities usually have a severity assigned.	low, medium, high, critical	4
	DETECTION	Vulnerabilities are detected manually or using specific tools.	Manual, CodeQL, Coverity, OSS-Fuzz, lib-fuzzer	8
COM	SHA	Commit hashes that reference older versions where the vulnerability was introduced (OSV Schema [31]).	f8d773084564, 228a782c2dd0	5
	ACTION	A commit usually implies an action, in the case of security, fixing a vulnerability (corrective maintenance).	fix, patch, change, add, remove, found, protect, update, optimize, mitigate	16
	FLAW	Fixing a security vulnerability usually implies fixing a flaw.	defect, weakness, flaw, fault, bug, issue	10
	ISSUE	The GitHub issue/pull request number is sometimes referenced in the message and can provide more information on the vulnerability.	#2, #13245	1
	EMAIL	Contact e-mails of reviewers and authors usually appear after tags such as 'Reported-by' and are important to know who to contact.	johndoe@gmail.com,johndoe@yahooo.com,johndoe@hotmail.com,johndoe@outlook.com ¹	1
	URL	Links to reports, blog posts, and bug-trackers references provide more information about the vulnerability.	https://www.htbridge.ch/advisory/multiple_vulnerabilities_in_mantisbt.html	1
	VERSION	Software versions are commonly referenced in commit messages.	3.1.0, v3.2, v2.6.28, 1.6.3, 2.1.395	3

Rule Examples

TABLE III: NER rule examples

ID	Label	Rule
R1	ACTION	<code>{"label": "ACTION", "pattern": [{"LOWER": {"REGEX": "fix.*"}, "POS": "VERB"}]}</code>
R2	VULNID	<code>{"label": "VULNID", "pattern": [{"LOWER": "osv"}, {"IS_PUNCT": true}, {"SHAPE": "dddd"}, {"IS_PUNCT": true}, {"LIKE_NUM": true}], "id": "CVE"}</code>

Targets all inflected forms of fix (e.g., “fix”, “fixing”, “fixed”, “fixes”) by matching tokens labeled as VERB during POS tagging.

Extracts all OSV vulnerability IDs (e.g, OSV-2023-27). We built rules for all the ecosystems PyPI, NPM, RubyGems, etc.

All the rules are available in our replication package at `labeller/entity_ruler/patterns.jsonl`.

Best Practices

Following best practices is a recognized strategy for producing clear, effective commit messages.

Drawing on 5 sources, **we identified 8 core recommendations** and developed corresponding compliance checks, supported by our NER technique.

In this study, we only focus on **structural aspects** yet.

TABLE V: Commit Message Best Practices

ID	Common Practice	Standard
C1	The header should be prefixed with a type.	[21]
C2	The message should have a one-line header/subject.	[21], [22], [24]
C3	The message should have a body.	[22], [24]
C4	The message should mention the author's contact (signed-off-by and authored-by).	[22], [24]
C5	The message should mention the reviewer's contact (reviewed-by).	[22], [24]
C6	The message should mention the reporter's contact (reported-by).	[22]
C7	The message should mention references to issues or pull requests.	[24]
C8	The message should include references to bug trackers.	[24]

All compliance checks, including additional ones not presented here, are available in our replication package.

R1. Which types of information can be extracted from security-related commit messages?

38.8% of commit messages did not include any security-related terms – making it harder to understand the vulnerability and patch context.

Only 12.1% referenced a Vulnerability ID – hindering effective patch tracking and linking to official reports.

Only 1.1% mention Severity and **0.2%** mention the Weakness ID, limiting the ability to prioritize patches based on risk.

Key security metadata is often absent or inconsistently documented!

TABLE IV: Extraction Results (RQ1)

Extraction Results for security-specific (SEC)			
Category	#Entities	#Commits	% Commits
SECWORD	16167	6757	61.2%
VULNID	1799	1330	12.1%
DETECTION	629	374	3.4%
SEVERITY	142	118	1.1%
CWEID	27	25	0.2%
Total	18766	7316	66.3%
Extraction Results for commit-specific (COM)			
Category	#Entities	#Commits	% Commits
ACTION	12340	7188	65.1%
EMAIL	4738	2087	18.9%
SHA	4791	1473	13.4%
FLAW	4236	2761	25.0%
ISSUE	3561	2805	25.4%
URL	1182	930	8.4%
VERSION	658	571	5.2%
Total	31505	9240	83.7%
Extraction Results for security- and commit-specific (SEC & COM)			
Category	#Entities	#Commits	% Commits
Total	50271	10312	94.4%

RQ2. How do security engineers apply best practices when writing security commit messages?

While 100% of commits have a header, **only 43.53%** use security-relevant or action-oriented terms.

Bodies are **missing in 47.44%** of messages.

Sign-offs, reporter, and reviewer information are **included in less than 20% of cases.**

Structural practices are inconsistently followed.

Standardization

SECOM

1st comprehensive standard for creating clear, structured, and **security-focused commit message**.

Constructed based on (1) the findings from RQ1 and RQ2, (2) established best practices for commit writing, and (3) feedback from the open-source security community.

Listing 1: SECOM Convention

```
vuln-fix : <header / subject> (<Vuln-ID>)  
  
<body>  
# What:      Describe the vulnerability.  
# Why:       Explain its impact or risk.  
# How:       Describe how it was fixed.  
</body>  
  
[For each identified weakness:]  
Weakness:    <Weakness Name or CWE-ID>  
Severity:    <Low | Medium | High | Critical>  
CVSS:        <CVSS Score (0-10)>  
Detection:   <Detection Method>  
Report:      <Link to Advisory or Report>  
Introduced-in: <Commit Hash of Introduced Vuln>  
[End Weakness Block]  
  
Reported-by:  <Name> (<Contact>)  
Reviewed-by:  <Name> (<Contact>)  
Co-authored-by: <Name> (<Contact>)  
Signed-off-by: <Name> (<Contact>)  
  
Bug-tracker: <Bug Tracker URL>  
Resolves:    <Issue or PR Number>  
See also:    <Related Issue or PR Number>
```


Standardization

SECOM

1st comprehensive standard for creating clear, structured, and **security-focused commit message**.

Constructed based on (1) the findings from RQ1 and RQ2, (2) established best practices for commit writing, and (3) feedback from the open-source security community.

5 main sections

- Header
- Body
- Metadata
- Contacts
- References

Listing 1: SECOM Convention

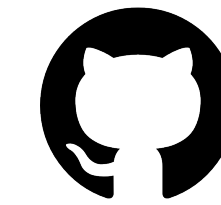
```
vuln-fix : <header / subject> (<Vuln-ID>)
```

```
<body>
# What:    Describe the vulnerability.
# Why:     Explain its impact or risk.
# How:     Describe how it was fixed.
</body>
```

```
[For each identified weakness:]
Weakness:  <Weakness Name or CWE-ID>
Severity:  <Low | Medium | High | Critical>
CVSS:      <CVSS Score (0-10)>
Detection: <Detection Method>
Report:    <Link to Advisory or Report>
Introduced-in: <Commit Hash of Introduced Vuln>
[End Weakness Block]
```

```
Reported-by:    <Name> (<Contact>)
Reviewed-by:    <Name> (<Contact>)
Co-authored-by: <Name> (<Contact>)
Signed-off-by:  <Name> (<Contact>)
```

```
Bug-tracker: <Bug Tracker URL>
Resolves:    <Issue or PR Number>
See also:    <Related Issue or PR Number>
```



Standardization

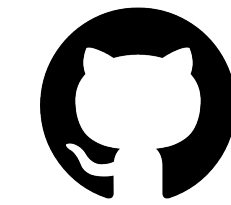
SECOM

Presented and validated at two OpenSSF working groups—*best practices* and *vulnerability disclosure*.

Refined with feedback collected from experts through a form—e.g., we picked the work “vuln-fix” for type as a suggestion from the Google OSV Team.

Listing 1: SECOM Convention

```
vuln-fix : <header / subject> (<Vuln-ID>)  
  
<body>  
# What:      Describe the vulnerability.  
# Why:      Explain its impact or risk.  
# How:      Describe how it was fixed.  
</body>  
  
[For each identified weakness:]  
Weakness:    <Weakness Name or CWE-ID>  
Severity:    <Low | Medium | High | Critical>  
CVSS:        <CVSS Score (0-10)>  
Detection:   <Detection Method>  
Report:      <Link to Advisory or Report>  
Introduced-in: <Commit Hash of Introduced Vuln>  
[End Weakness Block]  
  
Reported-by:  <Name> (<Contact>)  
Reviewed-by:  <Name> (<Contact>)  
Co-authored-by: <Name> (<Contact>)  
Signed-off-by: <Name> (<Contact>)  
  
Bug-tracker: <Bug Tracker URL>  
Resolves:    <Issue or PR Number>  
See also:    <Related Issue or PR Number>
```

Standardization

SECOM

Presented and validated at two OpenSSF working groups—*best practices* and *vulnerability disclosure*.

Refined with feedback collected from experts through a form—e.g., we picked the work “vuln-fix” for type as a suggestion from the Google OSV Team.

👍 Members of OpenSSF showed interest to apply and advocate for the convention inside their companies.

🤔 Some concerns about adoption were raised, particularly regarding the additional effort required—[motivating our exploration of LLMs to support the process](#).

Listing 1: SECOM Convention

```
vuln-fix : <header / subject> (<Vuln-ID>)

<body>
# What:      Describe the vulnerability.
# Why:      Explain its impact or risk.
# How:      Describe how it was fixed.
</body>

[For each identified weakness:]
Weakness:    <Weakness Name or CWE-ID>
Severity:    <Low | Medium | High | Critical>
CVSS:        <CVSS Score (0-10)>
Detection:   <Detection Method>
Report:      <Link to Advisory or Report>
Introduced-in: <Commit Hash of Introduced Vuln>
[End Weakness Block]

Reported-by:  <Name> (<Contact>)
Reviewed-by:  <Name> (<Contact>)
Co-authored-by: <Name> (<Contact>)
Signed-off-by: <Name> (<Contact>)

Bug-tracker:  <Bug Tracker URL>
Resolves:     <Issue or PR Number>
See also:     <Related Issue or PR Number>
```


Study Methodology

RQ3 (Standardization)



Analyzed 2,207 commit messages—20% of the dataset used to answer RQ1 and RQ2.



We asked LLaMa 3.2 to generate the commit message following the SECOM convention based on the code diff. **Prompts are available in our replication package.**



We accessed the number of the different entity categories that were extracted.

Using a short version of SECOM

```
prompt = f"""Given the following git diff, generate a
security commit message that adheres to the SECOM standard.
The <short description> describes a short summary of the
patch (max 72 chars). The <long description> describes in
3 sentences the vulnerability/problem (what), its impact
(why) and the patch/fix (how). Short and long descriptions
can not include code. Generate the commit message following
the standard. Note: Do not include any comments, explanations,
or surrounding text. Just return the message content as-is.
Remove any markdown signs.

Here is the SECOM standard:
...
vuln-fix: <short description> (Vuln-ID)

<long description>

Weakness: <CWE-ID>
Severity: <Low, Medium, High, Critical>
CVSS : <Numerical Representation of Severity: 0-10>
...

Here is the git diff:
...
""" + diff + """
..."""
```

RQ3. How do LLMs influence the completeness of security commit messages?

TABLE VII: Extraction Results for RQ3 (OR-original, MX-mixed_approach, FLLM-full_llm_generated)

Category	OR-#ENT	OR-#COM	OR-% COM	MX-#ENT	MX-#COM	MX-% COM	FLLM-#ENT	FLLM-#COM	FLLM-% COM
SECWORD	3325	1340	26.8%	13756	2107	42.1%	11292	2057	41.1%
ACTION	2399	1404	28.1%	6388	1984	39.7%	2437	1104	22.1%
FLAW	946	564	11.3%	3890	1831	36.6%	5111	1819	36.4%
VULNID	331	252	5.0%	2212	2187	43.7%	831	816	16.3%
SEVERITY	20	18	0.4%	1850	1807	36.1%	1334	1288	25.8%
CWEID	4	4	0.1%	1929	1770	35.4%	1667	1378	27.6%
Total	7025	2061	93.4%	30025	2207	100%	22672	2201	99.7%

BASELINE

**GENERATE HEADER + BODY +
KEEP VULN
REPORT METADATA**

**GENERATE THE ENTIRE
MESSAGE**

LLMs can greatly improve the completeness of commit messages when combined with real metadata.

RQ3. How do LLMs influence the completeness of security commit messages?

TABLE VII: Extraction Results for RQ3 (OR-original, MX-mixed_approach, FLLM-full_llm_generated)

Category	OR-#ENT	OR-#COM	OR-% COM	MX-#ENT	MX-#COM	MX-% COM	FLLM-#ENT	FLLM-#COM	FLLM-% COM
SECWORD	3325	1340	26.8%	13756	2107	42.1%	11292	2057	41.1%
ACTION	2399	1404	28.1%	6388	1984	39.7%	2437	1104	22.1%
FLAW	946	564	11.3%	3890	1831	36.6%	5111	1819	36.4%
VULNID	331	252	5.0%	2212	2187	43.7%	831	816	16.3%
SEVERITY	20	18	0.4%	1850	1807	36.1%	1334	1288	25.8%
CWEID	4	4	0.1%	1929	1770	35.4%	1667	1378	27.6%
Total	7025	2061	93.4%	30025	2207	100%	22672	2201	99.7%

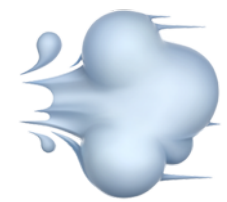
BASELINE

GENERATE HEADER + BODY +
KEEP VULN
REPORT METADATA

GENERATE THE ENTIRE
MESSAGE

LLMs can greatly improve the completeness of commit messages when combined with real metadata. **But relying solely on LLMs risks hallucinations.** – our current work focuses on semantic aspects.

Implications & Future Work



Better commit messages = faster patch deployment
SECOM



SECOM standard + LLMs = practical solution



Automated SECOM validation and generation tools
SECOMlint

**Read our paper to learn more about ethical considerations
and guidance for SECOM application.**

THANK YOU!

Motivation

TABLE I: Examples of Real Security Commit Messages

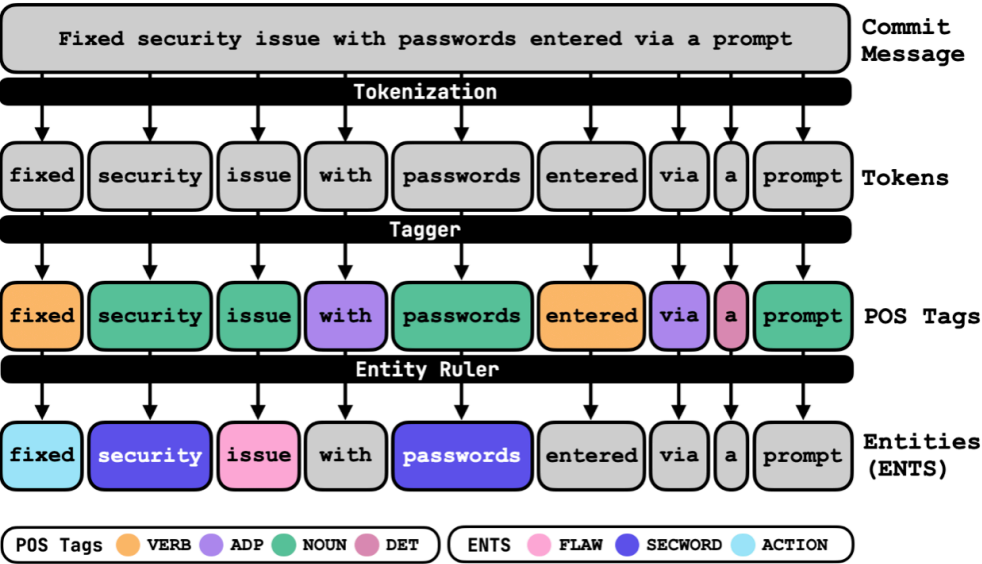
#	Commit Message	Vulnerability ID	SHA
1	.	CVE-2019-13568	ac80033
2	Minor patch.	OSV-2020-2108	a8bf10e
3	Code refactoring.	GHSA-4fc4-4p5g-6w89	d158413
4	<scratchsig><script>location='https://www.youtube.com/watch?v=dQw4w9WgXcQ';</script></scratchsig>	CVE-2020-15179	4160a39
5	Fixed security issues with passwords entered via a prompt	CVE-2022-35928	6876185

Message 4 is an internet meme known as "Rickrolling," where people are pranked with the song "Never Gonna Give You Up"—humorous, but it provides no relevant information about the patch.

NER for Security and Commit Data

Security Specific Categories
- SECWORD, VULNID, CWEID, SEVERITY, DETECTION

Commit Specific Categories
- SHA, ACTION, FLAW, ISSUE, EMAIL, URL, VERSION



@sofiaoreis

Stay updated and access the datasets and full paper here



Standardization

SECOM

1st comprehensive standard for creating clear, structured, and **security-focused commit message**.

Constructed based on (1) the findings from RQ1 and RQ2, (2) established best practices for commit writing, and (3) feedback from the open-source security community.

5 main sections

- Header
- Body
- Metadata
- Contacts
- References

Listing 1: SECOM Convention

vuln-fix:	<header/subject> (<Vuln-ID>)
<body>	
# What:	Describe the vulnerability.
# Why:	Explain its impact or risk.
# How:	Describe how it was fixed.
</body>	
[For each identified weakness:]	
Weakness:	<Weakness Name or CWE-ID>
Severity:	<Low Medium High Critical>
CVSS:	<CVSS Score (0-10)>
Detection:	<Detection Method>
Report:	<Link to Advisory or Report>
Introduced-in:	<Commit Hash of Introduced Vuln>
[End Weakness Block]	
Reported-by:	<Name> (<Contact>)
Reviewed-by:	<Name> (<Contact>)
Co-authored-by:	<Name> (<Contact>)
Signed-off-by:	<Name> (<Contact>)
Bug-tracker:	<Bug Tracker URL>
Resolves:	<Issue or PR Number>
See also:	<Related Issue or PR Number>

Read our paper to learn more about each field and rationale for inclusion.

@sofiaoreis

RQ3. How do LLMs influence the completeness of security commit messages?

TABLE VII: Extraction Results for RQ3 (OR-original, MX-mixed_approach, FLLM-full_llm_generated)

Category	OR-#ENT	OR-#COM	OR-%COM	MX-#ENT	MX-#COM	MX-%COM	FLLM-#ENT	FLLM-#COM	FLLM-%COM
SECWORD	3325	1340	26.8%	13756	2107	42.1%	11292	2057	41.1%
ACTION	2399	1404	28.1%	6388	1984	39.7%	2437	1104	22.1%
FLAW	946	564	11.3%	3890	1831	36.6%	5111	1819	36.4%
VULNID	331	252	5.0%	2212	2187	43.7%	831	816	16.3%
SEVERITY	20	18	0.4%	1850	1807	36.1%	1334	1288	25.8%
CWEID	4	4	0.1%	1929	1770	35.4%	1667	1378	27.6%
Total	7025	2061	93.4%	30025	2207	100%	22672	2201	99.7%

BASELINE

GENERATE HEADER + BODY +
KEEP VULN
REPORT METADATA

GENERATE THE ENTIRE
MESSAGE

LLMs can greatly improve the completeness of commit messages when combined with real metadata. **But relying solely on LLMs risks hallucinations.** – our current work focuses on semantic aspects.

@sofiaoreis

<https://security-commits.org/>

Save it for later, as we are still working on the website! 😊

@sofiaoreis